TP8 - Food Store en la feria

1.a Qué datos viajan a la tablet

La tablet recibe una instantánea del catálogo (vista `v_snapshot_movil`, exportada a CSV). Todo lo que no se usa para armar un borrador o evaluar una solicitud se queda en el servidor.

Dato	Viaja	Motivo
`id`	viaja	identifica el producto en el borrador
`nombre`	viaja	para mostrar el catálogo sin conexión
`precio_centavos`	viaja	precio actual, en enteros, para detectar conflictos
`stock`	viaja	para rechazar cantidades que no alcanzan
`disponible`	viaja	derivado: activo y con stock
`eliminado`	viaja	derivado: producto dado de baja (lápida)
`copiado_en`	viaja	fecha de la copia
`filas_copia`	viaja	total esperado, para verificar la importación
`cliente` completo	no viaja	datos personales: la tablet no los necesita
`created_at`, `categoria_id`	no viaja	no se usan en la tablet
pedidos y detalles del servidor	no viaja	son las ventas; las administra solo el servidor

La fuente de verdad es PostgreSQL: el precio, el stock y las ventas los decide el servidor. La tablet trabaja con una copia de solo lectura que puede estar vieja, y por eso cada solicitud se vuelve a comparar contra una instantánea nueva. Lo único que manda la tablet son sus borradores y solicitudes, y nada de eso vuelve al servidor en este TP.

2.c Casos de sincronización

3.a Mapa de los tres motores

Cada dato del ecosistema Food Store tiene un único motor que manda (fuente de verdad). Los demás motores mantienen copias derivadas de solo lectura o directamente no almacenan el dato.

Dato	PostgreSQL (Central)	MongoDB (U7)	SQLite (Tablet feria)
Catálogo y precio actual	**manda**	no está [1]	copia (solo lectura) en `producto_cache`
Stock	**manda**	no está	copia (solo lectura) (`stock_visto`)
Pedidos confirmados (ventas)	**manda**	copia (solo lectura): colección `foodstore.pedidos` (se regenera desde PostgreSQL)	no está
Estado del pedido	**manda** (`pedido.estado` en `foodstore`)	copia (solo lectura)	no está
Clientes y datos personales	**manda**	copia (solo lectura) del subconjunto esencial dentro del pedido [2]	no está (nunca viajan a la tablet)
Instantánea CSV (`snapshot_servidor`)	**manda** (vista `v_snapshot_movil`)	no está	copia (solo lectura)
Borradores (`pedido_borrador`, `item_borrador`)	no está	no está	**manda**

Dato	PostgreSQL (Central)	MongoDB (U7)	SQLite (Tablet feria)
Solicitudes de sincronización (`cola_sync`)	no está	no está	**manda** (la sincronización solo se simula)
Resultado del semáforo	no está	no está	**manda** (simulación local; `SIMULADO_APROBADO` nunca es una venta)

[1] Cada línea en MongoDB conserva el `precio_unitario` histórico del momento de la venta, no el precio actual del catálogo.

[2] En MongoDB se embeben los datos esenciales de cliente y producto; teléfono, dirección, activo, precio actual y stock no se embeben y pertenecen exclusivamente a PostgreSQL.

Nota de origen: MongoDB contiene 9 pedidos / 31 líneas migrados desde la base `foodstore` (TP7, que incluye `pedido.estado`); por su parte, la tablet trabaja sobre `food_store_copia`, copia de pruebas sin columna `estado`.

```
Flujo entre motores:
PostgreSQL (food_store_copia) ──[ v_snapshot_movil ]──► \copy ──► servidor_snapshot.csv ──► .import ──► SQLite (tablet)
PostgreSQL (foodstore) ──────────[ migracion TP7 ]──────► insertMany ──────────────────────────────► MongoDB (pedidos)
SQLite (tablet) ─────────────────[ X ]──────────────────► no vuelve nada al servidor (cola_sync solo simula)
```

3.b Plan de mantenimiento

El plan extiende el mantenimiento de la Unidad 6 (respaldo diario de PostgreSQL y prueba semanal) sin modificar el TP6:

- **Qué se respalda y restauración:**
  - *PostgreSQL:* Respaldo lógico diario con `pg_dump` y prueba de restauración semanal en servidor aislado.
  - *MongoDB:* No se respalda de forma autónoma. Se regenera desde PostgreSQL ejecutando la migración del TP7 (`insertMany`).
  - *SQLite:* Respaldo en caliente al cierre de cada jornada de feria con `VACUUM INTO 'respaldo_jornada_YYYYMMDD.db'`. La prueba de restauración se realiza semanalmente comprobando `PRAGMA integrity_check` y conteos en las cuatro tablas.
- **Contingencia ante restauración de respaldo viejo de PostgreSQL:** Las tablets no confían en su caché ni en resultados simulados previos. Invalidan `snapshot_servidor`, descargan una instantánea fresca con `copiado_en` posterior a la restauración del servidor y reevalúan las solicitudes `PENDIENTE` con el semáforo.
- **Revisión periódica:** Al cierre de cada jornada de feria (y antes de la siguiente sincronización), el **Encargado del Food Truck** revisa las solicitudes en `CONFLICTO` (para actualizar precio visto o desistir) y en `RECHAZADO` (para registrar mermas o anular).

3.c Code review con IA

Se realiz\u00f3 una revisi\u00f3n t\u00e9cnica de los 107 scripts de la cursada completa (TP1 a TP6, Proyecto Integrador, TP7 y TP8). **Declaraci\u00f3n de herramienta:** La revisi\u00f3n fue efectuada por un agente de Claude de solo lectura (modelo Claude Opus 5.5 / subagente `research`) y no por el agente Plan de OpenCode, garantizando trazabilidad y preservaci\u00f3n de comentarios.

ID	Unidad	Categor\u00eda	Decisi\u00f3n	Motivo
SEC-01	TP7	Seguridad	Descartado	Los campos provienen de ENUMs estrictamente alfanum\u00e9ricos en PostgreSQL (imposible inyecci\u00f3n).
SEC-02	TP7	Seguridad	Aceptado	Validaci\u00f3n de formato CSV en script de test local.
CORR-01	TP8	Correctitud	Descartado	La evaluaci\u00f3n local es simulaci\u00f3n (consistencia eventual); el stock real lo bloquea PostgreSQL.
CORR-02	TP7	Rendimiento	Aceptado	Filtrado temprano en vista comparativa evita escanear 621.853 filas para 9 pedidos (> 400x).
CORR-03	TP8	Correctitud	Descartado	El directorio en el host se llama literalmente `PosgresSQL`; corregir a ciegas romper\u00eda psql.
INTEG-01	TP8	Integridad	Aceptado	Exigencia de ejecutar `PRAGMA foreign_keys = ON;` en cada conexi\u00f3n cliente en SQLite.
INTEG-02	TP1\u20136	Integridad	Aceptado (hist\u00f3rico)	Falta de CHECK de subtotal en modelo relacional original (repositorio externo cerrado).
INTEG-03	TP3\u20134	Calidad	Aceptado (hist\u00f3rico)	Sesgo de 200.000 pedidos al cliente 4812 (lecci\u00f3n aprendida de generaci\u00f3n sint\u00e9tica).
INTEG-04	TP8	Integridad	Aceptado (fortaleza)	Validaci\u00f3n estricta de formato can\u00f3nico de UUIDs en tablas STRICT.

ID	Unidad	Categor\u00eda	Decisi\u00f3n	Motivo
IDEM-01	TP7	Idempotencia	Aceptado	Uso de `drop()` incondicional admisible solo en entorno de laboratorio.
IDEM-02	TP7	Rendimiento	Aceptado	UPDATE masivo en PG genera tuplas muertas; requiere VACUUM ANALYZE.
IDEM-03	TP8	Idempotencia	Aceptado (fortaleza)	Patr\u00f3n transaccional robusto con guarda temporal de filas y l\u00e1pidas en upsert.

(La tabla completa detallada se encuentra en `docs/code_review_ia.md`).

Tres hallazgos desarrollados con comprobaci\u00f3n emp\u00edrica:

1. **[CORR-01] Supuesta \"Aprobaci\u00f3n fantasma de stock\" en sem\u00e1foro de TP8 (Descartado con fundamento):** La IA cuestion\u00f3 que dos solicitudes encoladas de 2 unidades frente a un stock de 3 aprueben como simulaci\u00f3n. Se comprob\u00f3 en `evidencias/code_review_1.txt` que ambas aprueban porque `SIMULADO_APROBADO` es una propuesta desconectada monousuario en la feria, no una venta confirmada. Deducir stock localmente crear\u00eda inventarios divergentes entre tablets; la protecci\u00f3n ante sobreventa la ejerce PostgreSQL de forma at\u00f3mica al sincronizar.
2. **[CORR-02] Agrupaci\u00f3n exhaustiva en vista comparativa de TP7 (Aceptado):** En `consultas_postgresql.sql`, la vista temporal `pedido_total` agrupaba las 621.853 filas de `pedido_detalle`. Se comprob\u00f3 en `evidencias/code_review_2.txt` que agregar el filtro `WHERE d.pedido_id IN (SELECT id FROM migrados)` sustituye el escaneo completo por un Index Scan sobre 31 filas, reduciendo los buffers de 24.000 a 18 (< 0.2 ms).
3. **[CORR-03] Supuesto error tipogr\u00e1fico en ruta PosgresSQL en TP8 (Descartado con refutaci\u00f3n):** La IA recomend\u00f3 corregir la ruta `/d/DevTools/PosgresSQL/bin` a `PostgreSQL`. Se comprob\u00f3 en `evidencias/code_review_3.txt` inspeccionando el sistema de archivos del host que la carpeta instalada se llama exactamente `PosgresSQL`. Seguir la recomendaci\u00f3n a ciegas habr\u00eda roto el acceso a `psql` en la m\u00e1quina del usuario. Se configuraron ambas rutas en el PATH para asegurar portabilidad.